



UNAIR
HEBAT

WORLD CLASS UNIVERSITY
#521-530
QS TOP UNIVERSITY 2021

Pertemuan 13

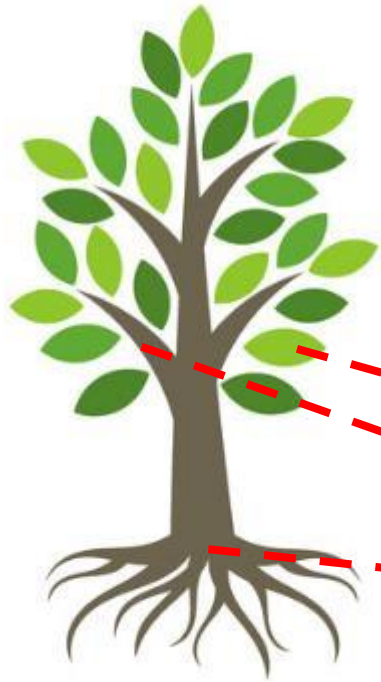
Tree

Muhammad Noor Fakhruzzaman, S.Kom., M.Sc.

MK Matematika Diskrit
Prodi S1 Teknologi Sains Data
Fakultas Teknologi Maju dan Multidisiplin
Universitas Airlangga



PENDAHULUAN



Pohon di dunia nyata

Apa itu Tree ?

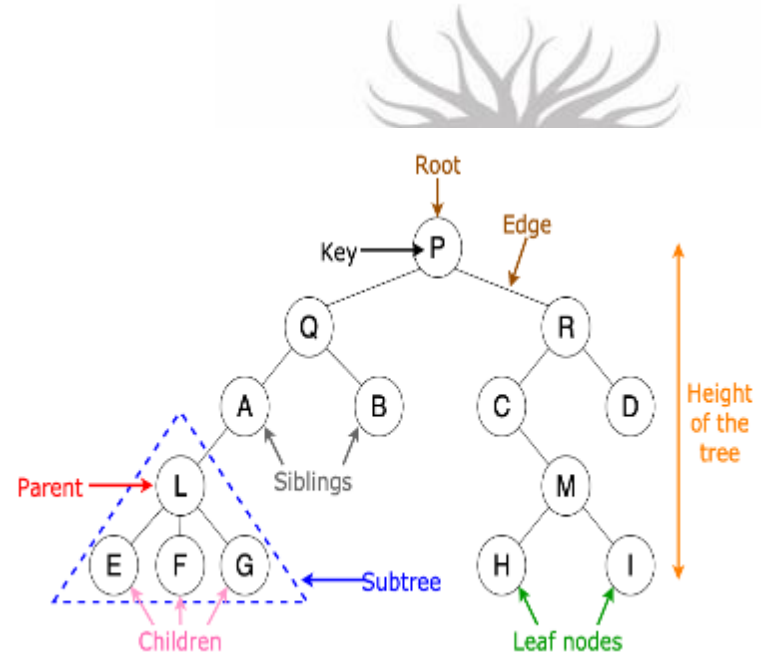
Dari arti katanya bisa disebut sebuah pohon (seperti dalam dunia nyata).

Tree adalah pemetaan data dalam bentuk hirarki seperti pohon, yang memiliki akar (*root*) dan cabang/ batang/ *subtree*.

Daun / Leaf Node

Cabang / Branch/ Edge

Akar / Root



Pohon dalam ilmu komputer

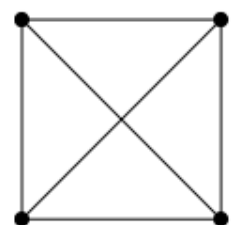
Teorema. Misalkan $G = (V, E)$ adalah graf tak-berarah sederhana dan jumlah simpulnya n . Maka, semua pernyataan di bawah ini adalah ekuivalen:

1. G adalah pohon.
2. Setiap pasang simpul di dalam G terhubung dengan lintasan tunggal.
3. G terhubung dan memiliki $m = n - 1$ buah sisi.
4. G tidak mengandung sirkuit dan memiliki $m = n - 1$ buah sisi.
5. G tidak mengandung sirkuit dan penambahan satu sisi pada graf akan membuat hanya satu sirkuit.
6. G terhubung dan semua sisinya adalah jembatan.

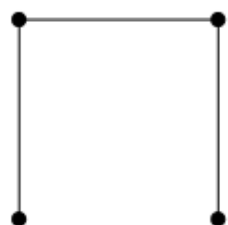
- Teorema di atas dapat dikatakan sebagai definisi lain dari pohon.

Pohon Merentang (*spanning tree*)

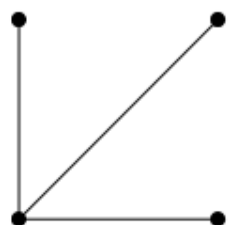
- Pohon merentang dari graf terhubung adalah upagraf merentang yang berupa pohon.
- Pohon merentang diperoleh dengan memutus sirkuit di dalam graf.



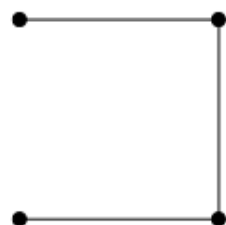
G



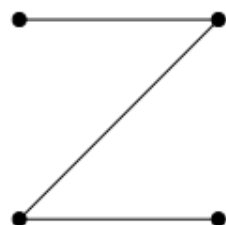
T_1



T_2



T_3

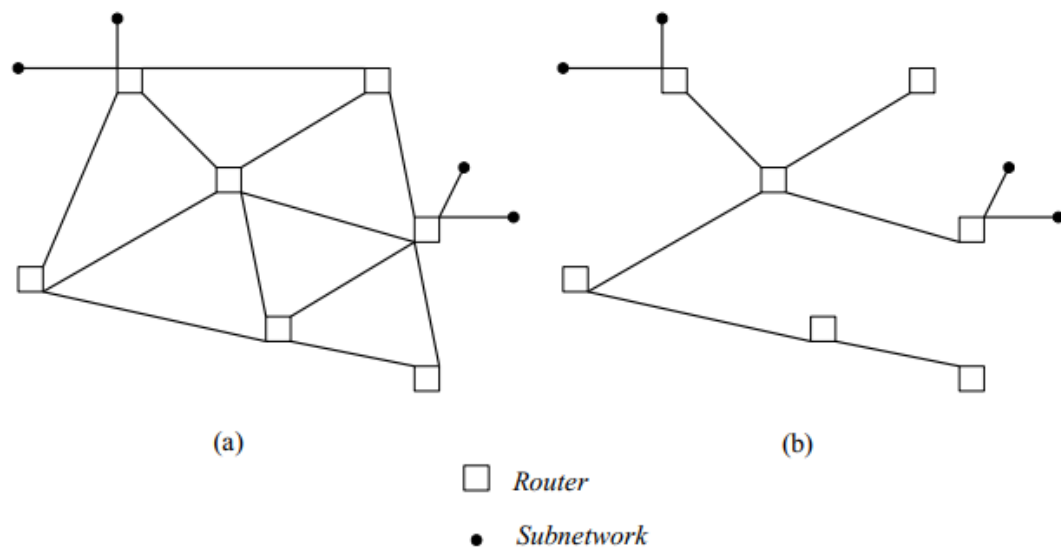


T_4

- Setiap graf terhubung mempunyai paling sedikit satu buah pohon merentang.
- Graf tak-terhubung dengan k komponen mempunyai k buah hutan merentang yang disebut hutan merentang (*spanning forest*).

Aplikasi Pohon Merentang

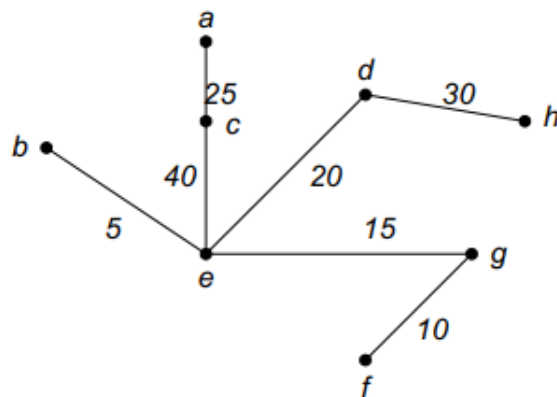
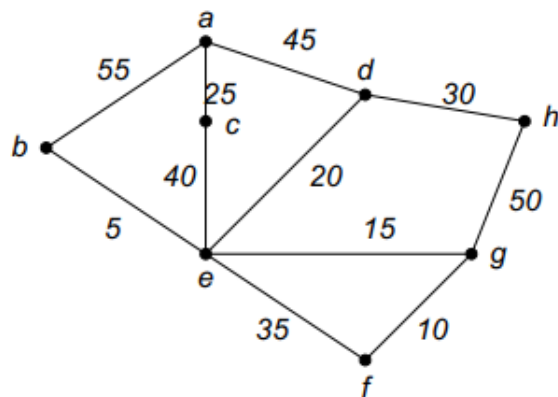
1. Jumlah ruas jalan semimumum mungkin yang menghubungkan semua kota sehingga setiap kota tetap terhubung satu sama lain.
2. Perutean (*routing*) pesan pada jaringan komputer.



(a) Jaringan komputer, (b) Pohon merentang *multicast*

Pohon Merentang Minimum

- Graf terhubung-berbobot mungkin mempunyai lebih dari 1 pohon merentang.
- Pohon merentang yang berbobot minimum –dinamakan **pohon merentang minimum** (*minimum spanning tree*).



Menentukan Pohon Merentang Minimum

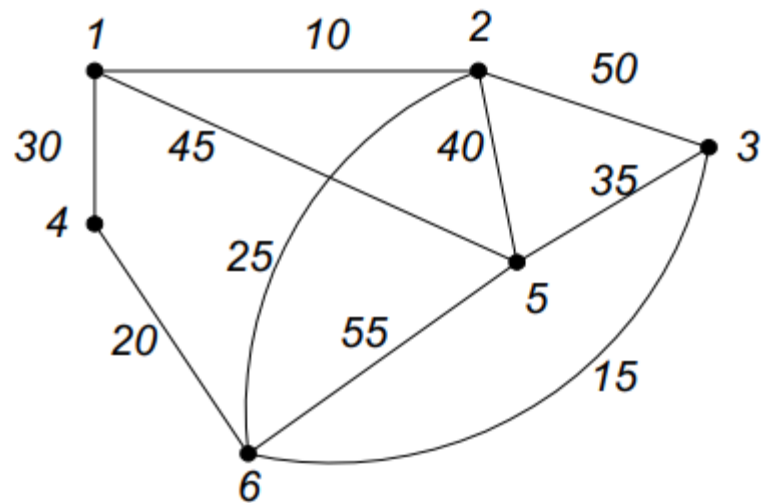
Algoritma Prim

Langkah 1: ambil sisi dari graf G yang berbobot minimum, masukkan ke dalam T .

Langkah 2: pilih sisi (u, v) yang mempunyai bobot minimum dan bersisian dengan simpul di T , tetapi (u, v) tidak membentuk sirkuit di T . Masukkan (u, v) ke dalam T .

Langkah 3: ulangi langkah 2 sebanyak $n - 2$ kali.

Contoh:

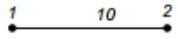
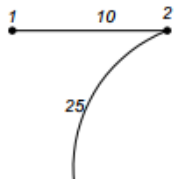
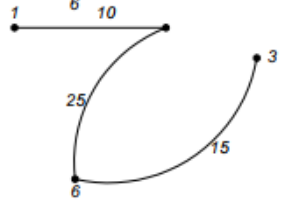
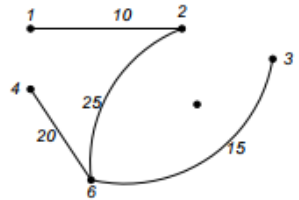
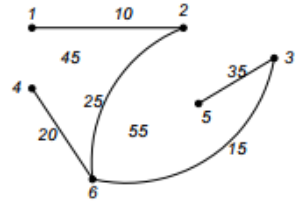


Algoritma Prim

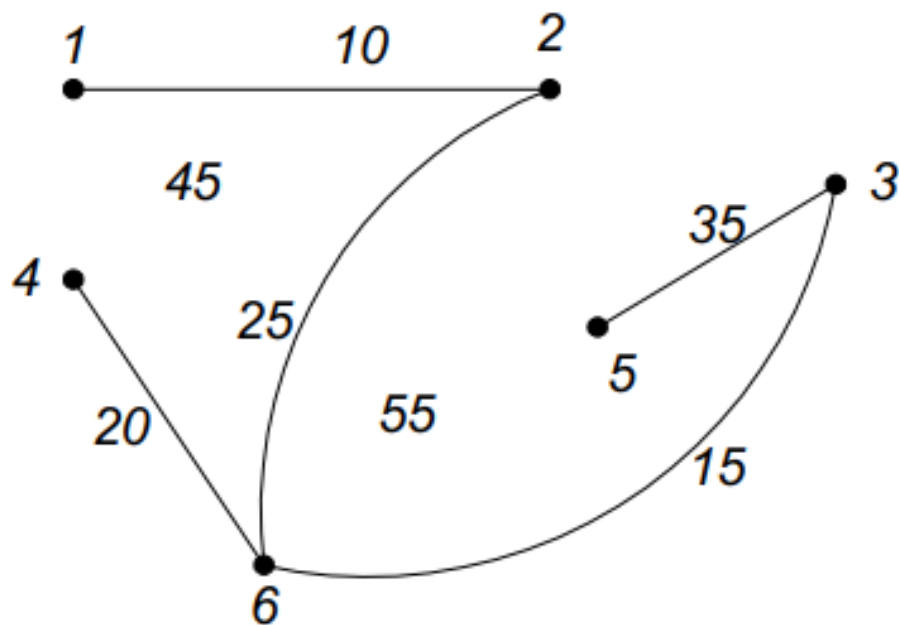
Langkah 1: ambil sisi dari graf G yang berbobot minimum, masukkan ke dalam T .

Langkah 2: pilih sisi (u, v) yang mempunyai bobot minimum dan bersisian dengan simpul di T , tetapi (u, v) tidak membentuk sirkuit di T . Masukkan (u, v) ke dalam T .

Langkah 3: ulangi langkah 2 sebanyak $n - 2$ kali.

Langkah	Sisi	Bobot	Pohon rentang
1	(1, 2)	10	
2	(2, 6)	25	
3	(3, 6)	15	
4	(4, 6)	20	
5	(3, 5)	35	

Pohon merentang minimum yang dihasilkan:



$$\text{Bobot} = 10 + 25 + 15 + 20 + 35 = 105$$

Algoritma Kruskal

(Langkah 0: sisi-sisi dari graf sudah diurut menaik berdasarkan bobotnya – dari bobot kecil ke bobot besar)

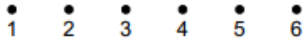
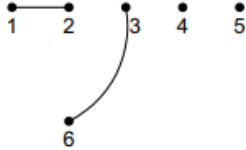
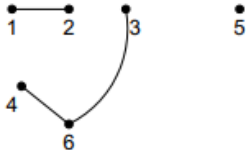
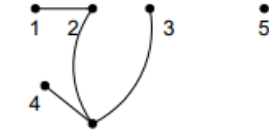
Langkah 1: T masih kosong

Langkah 2: pilih sisi (u, v) dengan bobot minimum yang tidak membentuk sirkuit di T . Tambahkan (u, v) ke dalam T .

Langkah 3: ulangi langkah 2 sebanyak $n - 1$ kali.

Sisi-sisi diurut menaik:

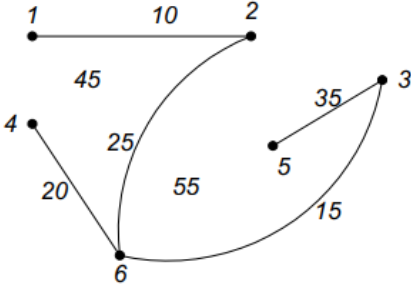
Sisi	(1,2)	(3,6)	(4,6)	(2,6)	(1,4)	(3,5)	(2,5)	(1,5)	(2,3)	(5,6)
Bobot	10	15	20	25	30	35	40	45	50	55

Langkah	Sisi	Bobot	Hutan merentang
0			
1	(1, 2)	10	
2	(3, 6)	15	
3	(4, 6)	20	
4	(2, 6)	25	

5 (1, 4) 30 ditolak



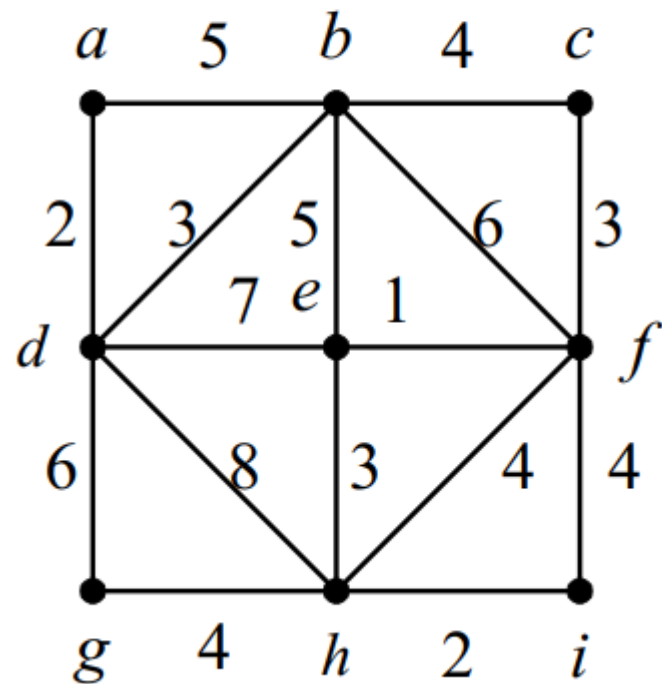
Pohon merentang minimum yang dihasilkan:



Bobot = 10 + 25 + 15 + 20 + 35 = 105

Latihan

Tentukan dan gambarkan pohon merentang minimum dari graf di bawah ini

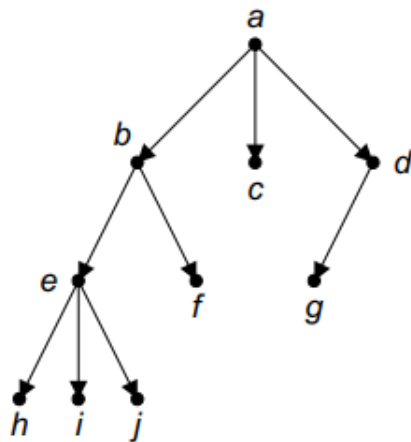


KONSEP STRUKTUR DATA TREE

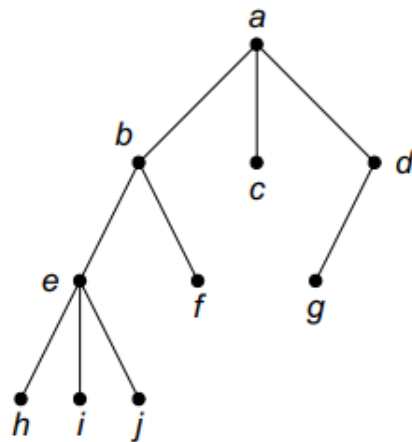
- Tree merupakan salah satu struktur data yang bersifat **non-linier**, artinya menggambarkan hubungan yang bersifat hirarki antara elemen-elemennya.
- Sesuai dengan penggambaran/pemetaannya, struktur *tree* ini berisi kumpulan elemen/node yang terhubung satu sama lain dalam satu kesatuan dan membentuk layaknya struktur sebuah pohon di dunia nyata.

Pohon berakar (*rooted tree*)

- Pohon yang satu buah simpulnya diperlakukan sebagai akar dan sisi-sisinya diberi arah sehingga menjadi graf berarah dinamakan **pohon berakar** (*rooted tree*).



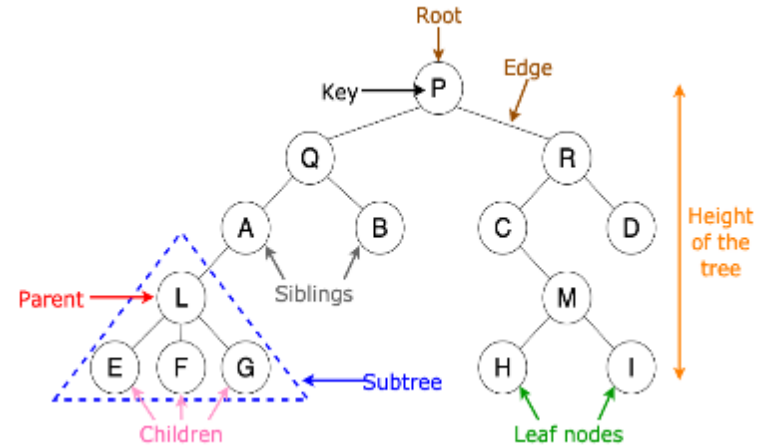
(a) Pohon berakar



(b) sebagai perjanjian, tanda panah pada sisi dapat diabaikan

KONSEP STRUKTUR DATA TREE

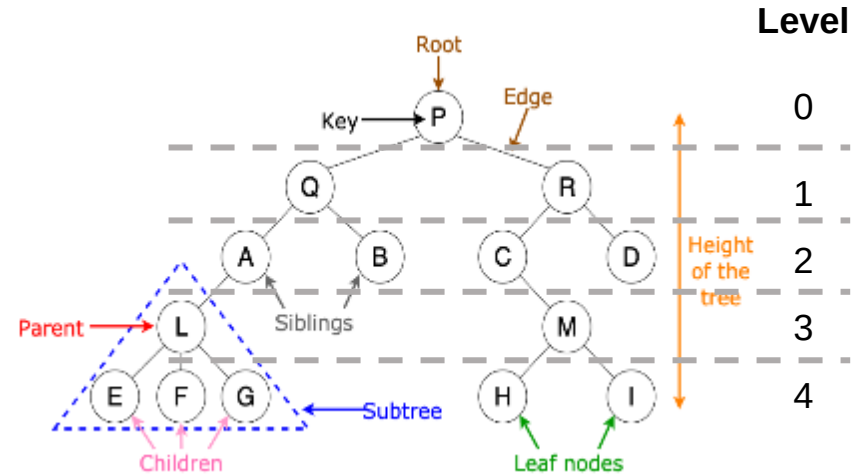
- Dalam konteks ilmu komputer (*computer science*), tree merupakan:
- Kumpulan elemen (*nodes*)
- Salah satu dari elemennya disebut sebagai *root* (akar)
- Sisanya disebut sebagai *node* (simpul)
- Membentuk hirarki kumpulan node dengan struktur dari **atas ke bawah**



KONSEP STRUKTUR DATA TREE

- **Node root**

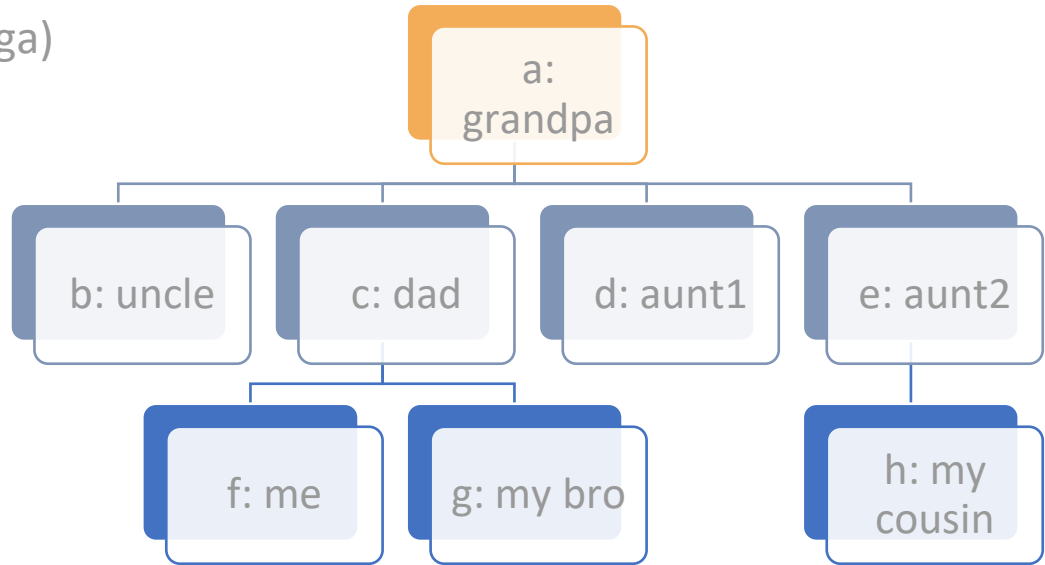
- Suatu node yang memiliki hirarki tertinggi dan dapat memiliki node-node anak (child)
- Node root tercipta pertama kali saat membentuk tree
- Semua node dapat ditelusuri dari node root tersebut
- Node-node lain yang berada di bawah root dan terhubung satu sama lain disebut sebagai *subtree*



CONTOH STRUKTUR DATA TREE

- Controh ilustrasi struktur pohon:

- Family Tree (Pohon Keluarga)
- Struktur Organisasi
- Jadwal Turnamen
- dll



TERMINOLOGI STRUKTUR DATA TREE

Predecessor	Node yang berada diatas node tertentu.
Successor	Node yang berada dibawah node tertentu.
Ancestor	Seluruh node yang terletak sebelum node tertentu dan terletak pada jalur yang sama
Descendant	Seluruh node yang terletak setelah node tertentu dan terletak pada jalur yang sama
Parent	Predecessor satu level di atas suatu node.
Child	Successor satu level di bawah suatu node.
Sibling	Node-node yang memiliki parent yang sama
Subtree	Suatu node beserta descendantnya.
Size	Banyaknya node dalam suatu tree
Height	Banyaknya tingkatan dalam suatu tree
Root	Node khusus yang tidak memiliki predecessor.
Leaf	Node-node dalam tree yang tidak memiliki successor.
Degree	Banyaknya child dalam suatu node

Path : urutan nodes yang menjadi jalur lintasan kunjungan pohon yang terhubung oleh edges

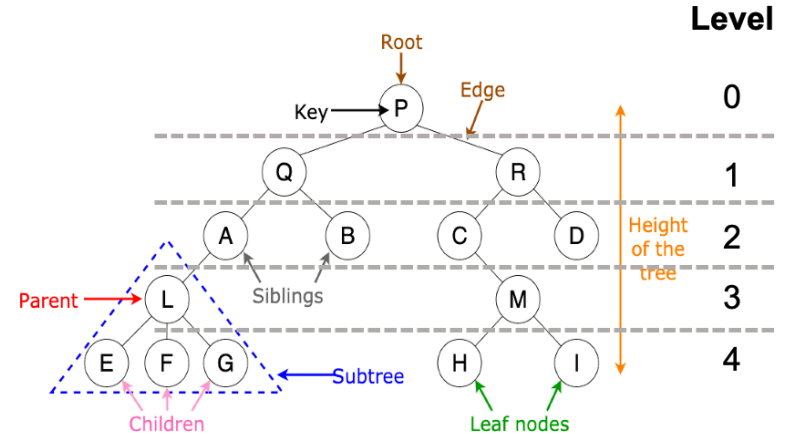
TERMINOLOGI STRUKTUR DATA TREE

- **Lintasan (path)** sebuah node X adalah urutan akses untuk mendapatkan X yang dimulai dari Akar.

- Contoh (dengan menggunakan tree di samping):

$\text{Path}(M) = [P - R - C - M]$

- **Panjang lintasan (path length)** X adalah jumlah node yang harus diakses untuk mendapatkan X; sehingga $|\text{Path}(M)| = 4$.



- **Kedalaman (depth)** X menyatakan panjang lintasan sebuah node adalah jumlah garis dari akar sampai mencapai node X tersebut.

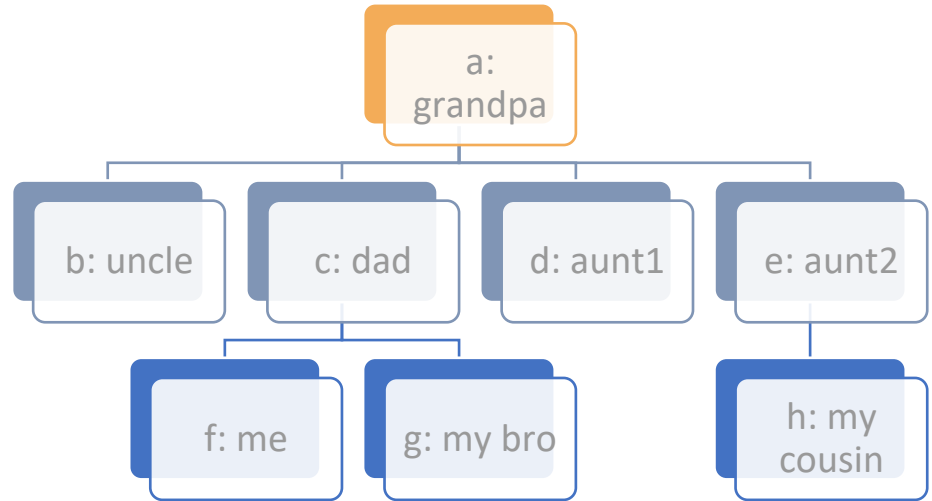
- Contoh:

$\text{Depth}(M) = 3$

IDENTIFIKASI TERMINOLOGI DALAM TREE

Contoh dari ilustrasi pohon keluarga:

- Ancestor (f) = c, a
- Descendant (c) = f, g
- Parent (h) = e
- Child (a) = b, c, d, e
- Sibling (d) = b, c, e
- Size = 8
- Height = 3
- Root = a
- Leaf = b, d, f, g, h
- Degree (e) = 1

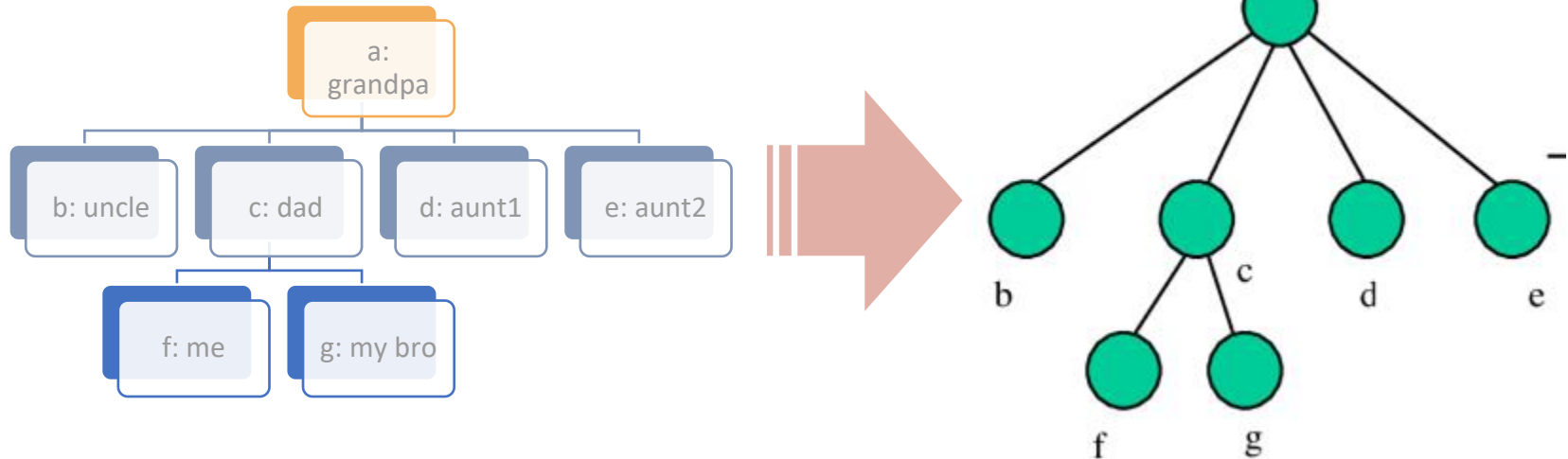


KARAKTERISTIK STRUKTUR DATA TREE

- Terdapat 1 *node* yang unik, yang tidak memiliki *predecessor*, yang disebut dengan *root* (akar)
- Terdapat satu atau beberapa *node* yang tidak memiliki *successor* yang disebut dengan *leaf* (daun)
- Setiap node kecuali *root* pasti memiliki satu *predecessor*
- Setiap node kecuali *leaf* pasti memiliki 1 atau lebih *successor*

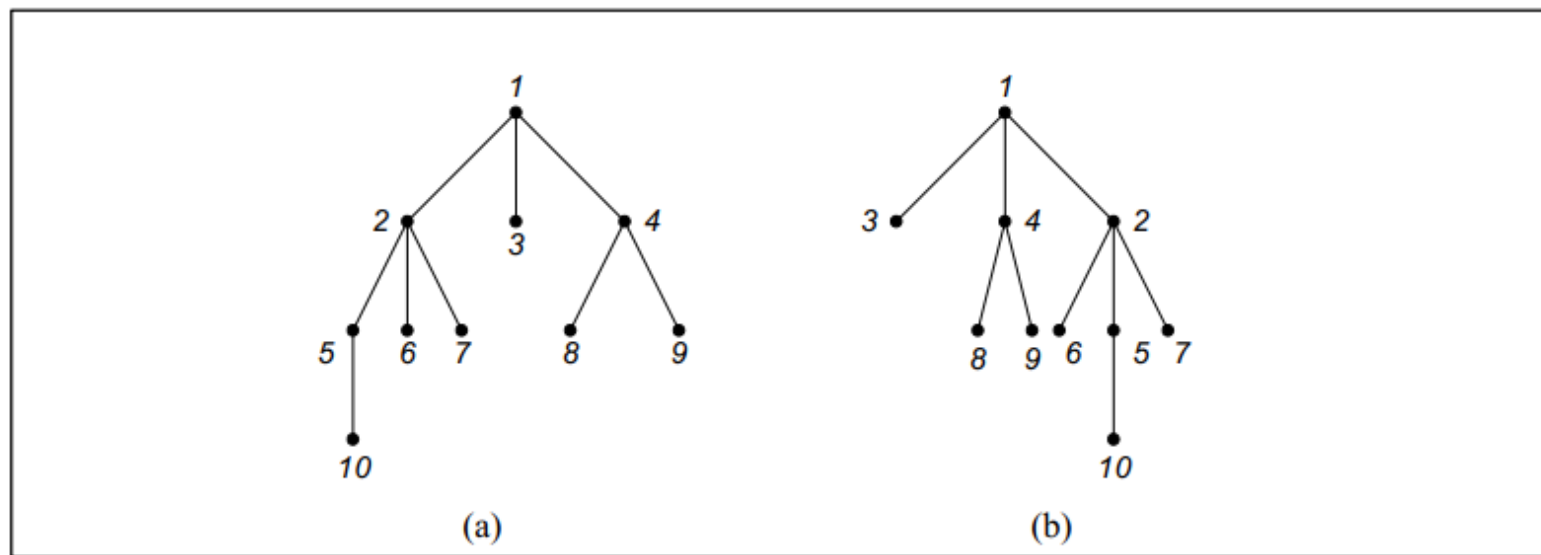
REPRESENTASI STRUKTUR DATA TREE

- Paling Umum: Pohon Berakar



Pohon Terurut (*ordered tree*)

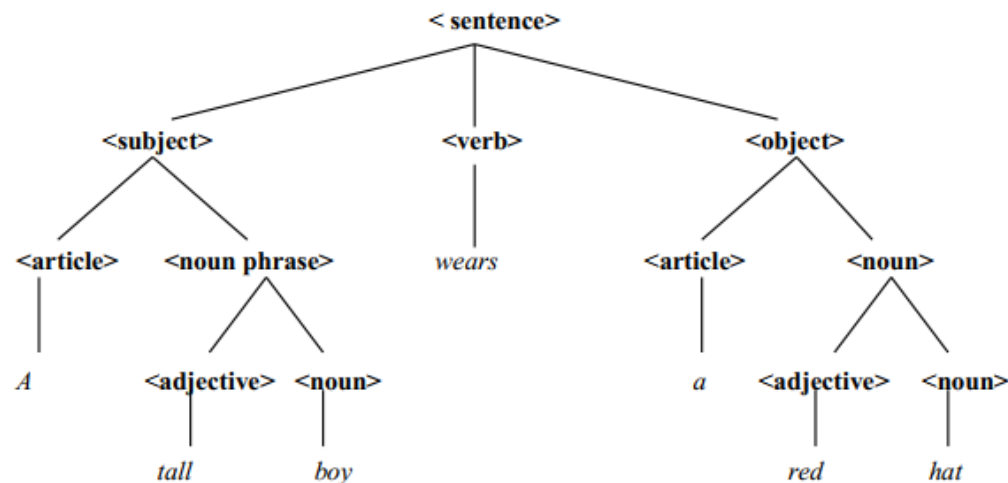
Pohon berakar yang urutan anak-anaknya penting disebut **pohon terurut** (*ordered tree*).



(a) dan (b) adalah dua pohon terurut yang berbeda

Pohon n -ary

- Pohon berakar yang setiap simpul cabangnya mempunyai paling banyak n buah anak disebut **pohon n -ary**.



Gambar Pohon parsing dari kalimat *A tall boy wears a red hat*

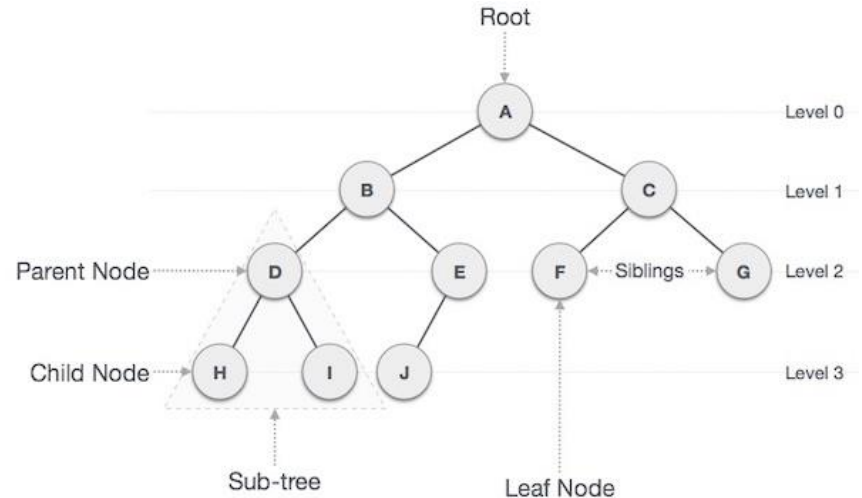
- Pohon n -ary dikatakan **teratur** atau **penuh (full)** jika setiap simpul cabangnya mempunyai tepat n anak.

Pohon Biner (*binary tree*)

- Adalah pohon *n-ary* dengan $n = 2$.
- Pohon yang paling penting karena banyak aplikasinya.
- Setiap simpul di adlam pohon biner mempunyai paling banyak 2 buah anak.
- Dibedakan antara anak kiri (*left child*) dan anak kanan (*right child*)
- Karena ada perbedaan urutan anak, maka pohon biner adalah pohon terurut.

KONSEP BINARY TREE

- **Binary tree** merupakan struktur data abstrak berjenis *tree* yang memiliki persyaratan:
 - **Binary tree** adalah sebuah struktur data abstrak berjenis *tree* dengan syarat satu *parent node* hanya dapat memiliki maksimal dua buah *child nodes*. Ilustrasi *binary tree* dapat dilihat seperti di samping.
 - Setiap node hanya boleh memiliki maksimal 2 buah subtrees, dan keduanya harus terpisah.



Latihan

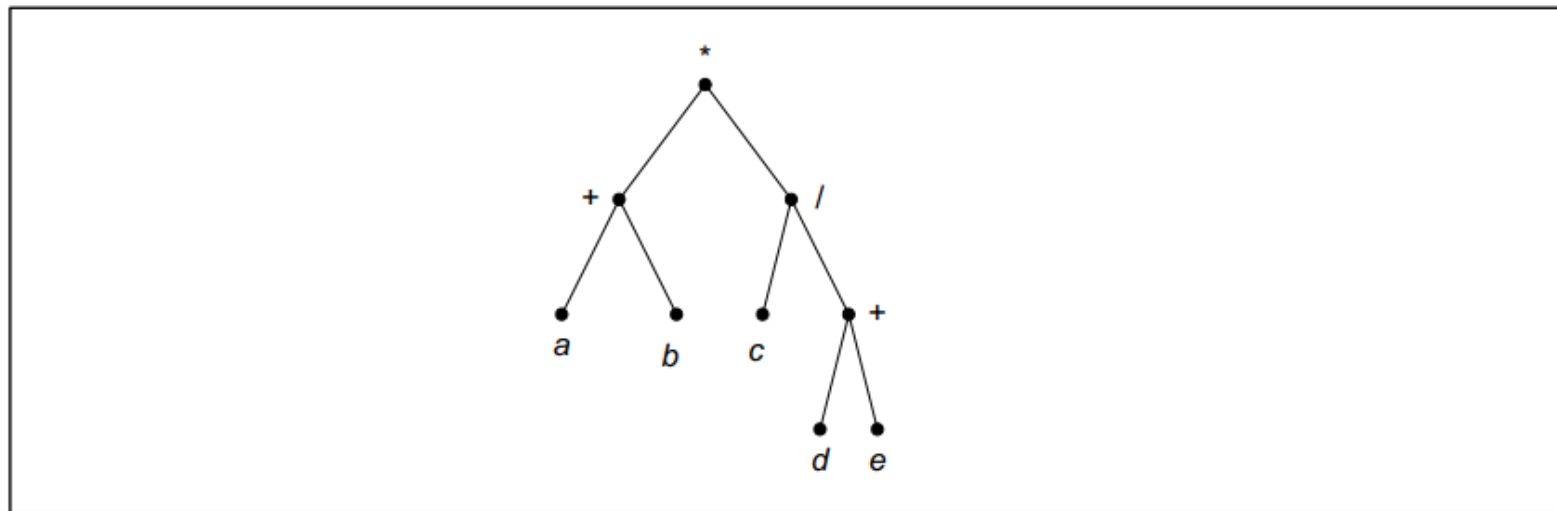
Diketahui 8 buah koin uang logam. Satu dari delapan koin itu ternyata palsu. Koin yang palsu mungkin lebih ringan atau lebih berat daripada koin yang asli. Misalkan tersedia sebuah timbangan neraca yang sangat teliti. Buatlah pohon keputusan untuk mencari uang palsu dengan cara menimbang paling banyak hanya 3 kali saja.



(Petunjuk: misalkan 8 koin tersebut adalah a, b, c, d, e, f, g, dan h)

Terapan Pohon Biner

1. Pohon Ekspresi

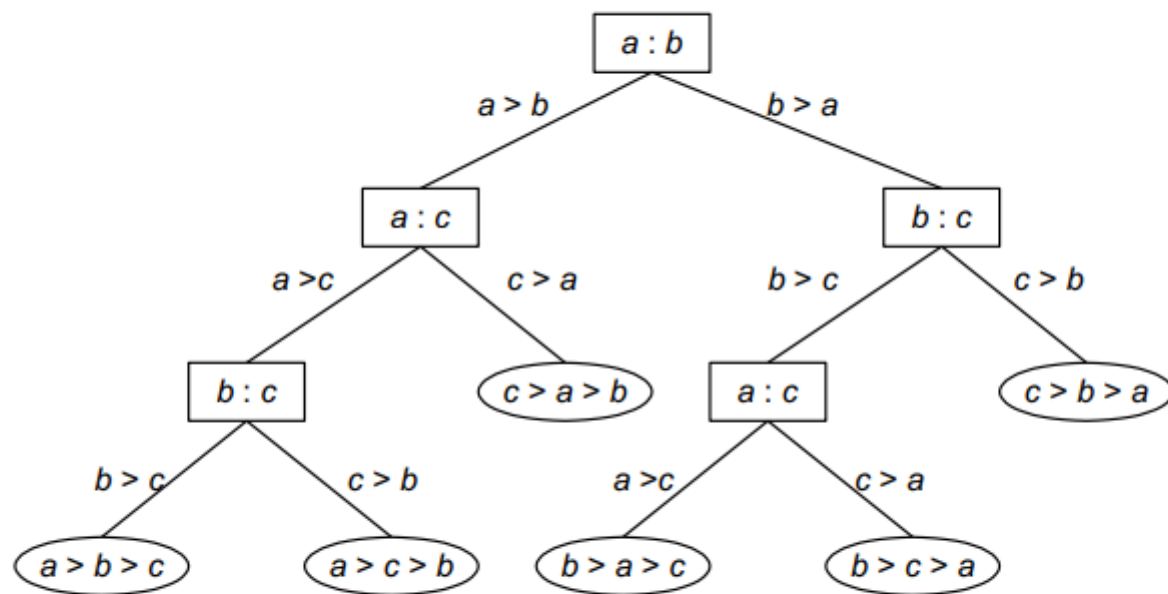


Gambar Pohon ekspresi dari $(a + b) * (c / (d + e))$

daun $\rightarrow$ *operand*

simpul dalam $\rightarrow$ operator

2. Pohon Keputusan

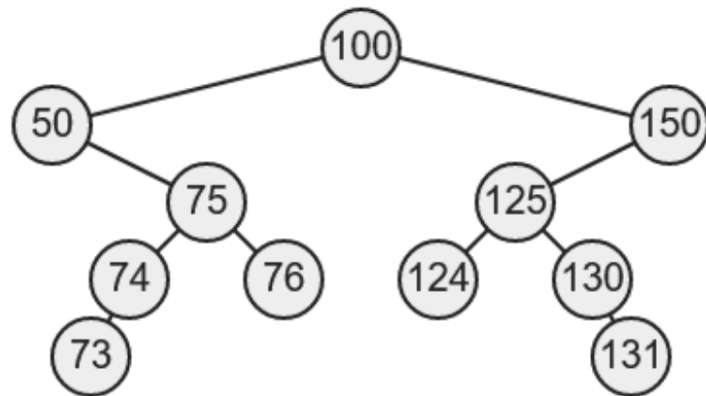


Gambar Pohon keputusan untuk mengurutkan 3 buah elemen

KONSEP BINARY SEARCH TREE

- Dalam *binary tree*, *child node* sebelah kiri hanya boleh bernilai **kurang dari atau sama dengan** *parent node*, dan *child node* sebelah kanan hanya boleh bernilai **lebih dari atau sama dengan** *parent node*.
- Dikenal dengan konsep ***Binary Search Tree***

Binary Search Tree Data Structure



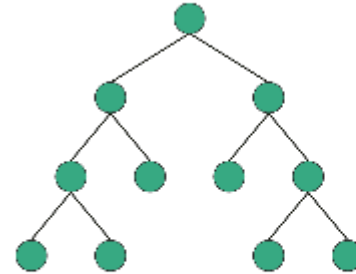
1. Nodes on left side lower than root

2. Nodes on right side higher than root

MACAM STRUKTUR BINARY TREE (1/2)

- **Full Binary Tree**

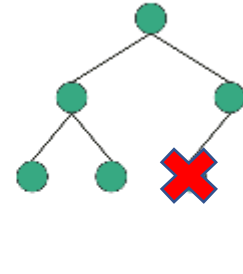
- Semua node (kecuali *leaf*) **PASTI** memiliki 2 buah *child*.



Full

- **Complete Binary Tree**

- Bila semua *node* kecuali *leaf* memiliki 0 atau 2 child. *Subtree* pada Tree **BOLEH** memiliki *path length* yang berbeda



Complete

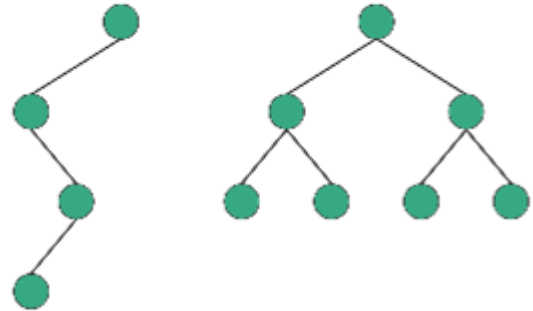
MACAM STRUKTUR BINARY TREE (2/2)

- **Perfect Binary Tree**

- Semua node (kecuali *leaf*) **PASTI** memiliki 2 buah *child* dan masing-masing *subtree* memiliki panjang *path* yang sama

- **Skewed/Degenerate Binary Tree**

- Bila semua *node* kecuali *Leaf*, hanya memiliki 1 *child*.

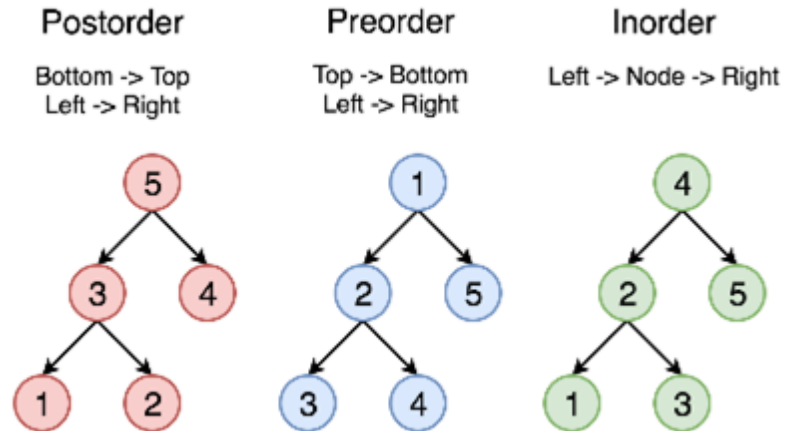


Degenerate

Perfect

POLA KUNJUNGAN POHON (TREE TRAVERSAL)

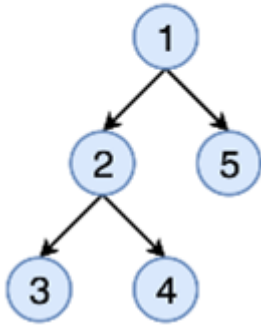
- Ada tiga pola umum yang digunakan untuk mengunjungi semua node dalam tree. Pola kunjungan tersebut dikenal dengan istilah **traversal** yang memiliki beberapa macam struktur lintasannya seperti tampak pada gambar di samping.
- Istilah **traversal** memiliki maksud penggambaran jalur lintasan penelusuran **SELURUH** nodes dalam *tree* dengan urutan/cara tertentu (*specific order*)



POLA KUNJUNGAN POHON (TREE TRAVERSAL)

Preorder

Top -> Bottom
Left -> Right



1. Preorder: (key: **root, kiri, kanan**)

Penelusuran node berawal dari ROOT, kemudian berlanjut ke turunan dari sebelah kiri, jika jalur kiri sudah tidak ada anak lagi, diteruskan ke jalur turunan sebelah kanan, terus berlanjut sampai akhir node. Atau pertama menunjuk ke node ROOT kemudian secara recursive melakukan preorder pada subtree sebelah kiri, diikuti dengan traversal preorder secara recursive ke subtree sebelah kanan.

Algorithm

Until all nodes are traversed –

Step 1 – Visit root node.

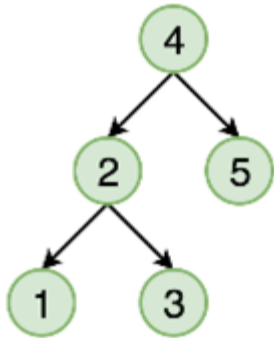
Step 2 – Recursively traverse left subtree.

Step 3 – Recursively traverse right subtree.

POLA KUNJUNGAN POHON (TREE TRAVERSAL)

Inorder

Left -> Node -> Right



- **Inorder:** (key: **kiri, root, kanan**)

Melakukan traversal inorder secara recursive pada subtree sebelah kiri, menuju ke node ROOT, dan terakhir melakukan traversal inorder secara recursive pada subtree sebelah kanan.

Algorithm

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

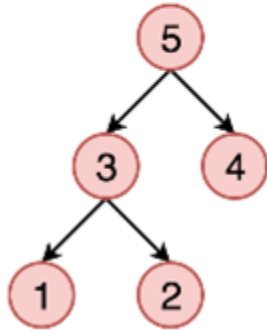
Step 2 – Visit root node.

Step 3 – Recursively traverse right subtree.

POLA KUNJUNGAN POHON (TREE TRAVERSAL)

Postorder

Bottom -> Top
Left -> Right



3. Postorder: (key: *kiri, kanan, root*)

Melakukan traversal postorder secara recursive pada subtree bagian kiri dan kanan diikuti dengan berkunjung ke node ROOT.

Algorithm

Until all nodes are traversed –

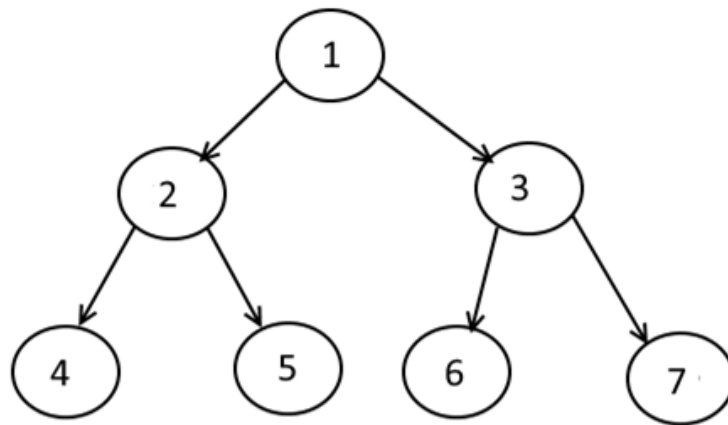
Step 1 – Recursively traverse left subtree.

Step 2 – Recursively traverse right subtree.

Step 3 – Visit root node.

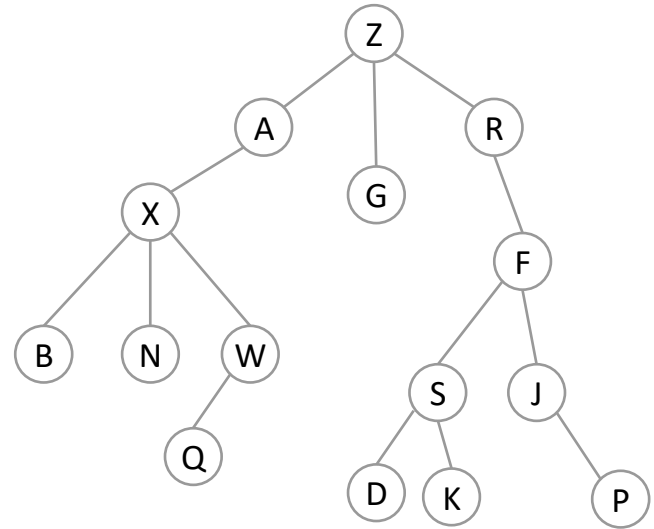
CONTOH TREE TRAVERSAL

- Dari *binary tree* di samping:
 - Inorder Traversal (*kiri, root, kanan*)
 - Hasil: 4, 2, 5, 1, 6, 3, 7
 - Preorder Traversal (*root, kiri, kanan*)
 - Hasil: 1, 2, 4, 5, 3, 6, 7
 - Postorder Traversal (*kiri, kanan, root*)
 - Hasil: 4, 5, 2, 6, 7, 3, 1



LATIHAN 1

- Tentukan penelusuran kunjungan *tree* dengan menggunakan metode inorder, preorder, dan postorder traversal



Latihan

Diberikan masukan berupa rangkaian karakter dengan urutan sebagai berikut:

P, T, B, F, H, K, N, S, A, U, M, I, D, C, W, O

- (a) Gambarkan pohon pencarian (*search tree*) yang terbentuk.
- (b) Tentukan hasil penelusuran *preorder*, *inorder*, dan *postorder*, dari pohon jawaban (a) di atas.

Terima Kasih